INTERVIEW PREPARATION

Software Engineer III — DevOps / SRE

Chevron (via HiyaMee Pvt Ltd) · Bengaluru · Contract-to-Hire

Client	Chevron
Payroll Company	HiyaMee Pvt Ltd
Location	Floor 3, Campus 32, RMZ Ecoworld Rd, Adarsh Palm Retreat, Bellandur, Bengaluru
Mode	Hybrid — Monday WFH, 4 days from office
Shift	1:30 PM – 10:30 PM IST
Coverage	123 questions across 14 topic areas

Prepared as a personal interview-study reference. Each item includes a checkbox to track review progress.

Table of Contents

01 Role & Positioning — 7 questions

02 CI/CD — Azure DevOps & GitHub — 12 questions

03 Scripting & Automation — 10 questions

04 VM Image Automation — 9 questions

05 BU Deployments & Hypercare — 8 questions

06 Testing & Quality — 9 questions

07 SRE — Monitoring & Incidents — 13 questions

08 Backend & Microservices — 7 questions

09 Data Platform & AI Enablement — 7 questions

10 Cloud — Azure, GCP & Multi-cloud — 8 questions

11 Networking & Security — 7 questions

12 Energy / Subsurface Context — 7 questions

13 Tricky & Scenario Questions — 9 questions

14 Leadership & Behavioral — 10 questions

01 Role & Positioning

1 group(s) · 7 questions

FRAMING THIS ENGAGEMENT

This is a contract-to-hire Software Engineer III role via HiyaMee at Chevron — how do you talk about that structure if asked directly?

Frame it factually and confidently: HiyaMee is the payroll/deployment partner, Chevron is the client I'd be embedded with day to day, and the engagement is structured as contract-to-hire — strong performance during the contract period is the path to a permanent Chevron offer. I'd say I look at this the same way I look at any role: what matters is the work (CI/CD, SRE, platform reliability for subsurface compute) and the team, not the payroll entity. This also mirrors part of my own background — I've worked through a payroll/staffing structure before, so the mechanics aren't new to me.

The JD asks for 8+ years but the title is "Software Engineer III" — how do you address a title/seniority mismatch if it comes up?

I'd acknowledge it plainly rather than dodge it: titles vary a lot across companies for the same scope of work, and what I'm looking at is the actual responsibilities — owning CI/CD pipelines, driving platform reliability, coordinating BU deployments, implementing SRE practices — which line up with senior IC/lead-level scope at my last few roles. I'd rather be evaluated on scope match than a label.

Why does an SRE/DevOps background make sense for a role sitting inside subsurface/exploration compute platforms at an oil & gas major, rather than a domain (geoscience) role?

This is explicitly a platform-engineering seat, not a subsurface-science seat — the JD asks for CI/CD, deployment automation, SRE practices, and data-platform enablement "to support AI, analytics, and business-critical workflows," not reservoir modeling expertise. My job would be to make the platform underneath the scientists and engineers reliable, fast to deploy, and observable — the same skill set I've applied in banking and on a large Azure data platform before, just a different vertical. Domain fluency is something I'd build on the job; the platform-reliability muscle is what I bring on day one.

How do you frame your Azure certifications (AZ-104, SC-300, SC-100) against a JD that lists Azure DevOps, GitHub, and Google Cloud?

AZ-104 is directly relevant — it's Azure infrastructure administration, which covers VM image management, storage, networking, and monitoring, all core to "automated VM image creation for compute workloads." SC-300/SC-100 round out identity and security architecture, which maps to the JD's networking-and-security-best-practices line. I'd position GitHub and Google Cloud as tools I'm comfortable picking up quickly given a strong Azure and general cloud-native foundation — the primitives (pipelines, artifact registries, IAM, compute) transfer directly across clouds.

The shift is 1:30 PM–10:30 PM IST — why would a Bengaluru-based Chevron team run that shift?

I'd read that as deliberate overlap with US hours — Chevron is Houston-headquartered, and a 1:30–10:30 PM IST shift gives several hours of overlap with US Central time in the morning there. For a role coordinating BU deployments and hypercare support, that overlap is probably the point — it lets the India team hand off, do live deployment support, and be reachable during US business hours rather than working fully offset. I'd confirm this in the interview rather than assume, but I'd frame it as a fit, not a friction point.

How would you describe your understanding of "PPO/FDO workflows" if asked, given the JD doesn't spell out the acronyms?

I'd be honest rather than bluff a definition — I'd say I read PPO and FDO as internal Chevron workflow/program names tied to portfolio and field-development-type processes based on the JD's context, but I wouldn't guess at the precise scope in the interview. My job is the same regardless of exactly what they stand for: give whatever workflow sits on top of subsurface compute a reliable CI/CD path, monitoring, and deployment automation — and I'd ask them to walk me through the actual workflow so I can map my answer to their real systems.

Contract-to-hire with a stated LWD of 22 July 2026 on your side — how do you handle a notice-period question in this interview?

Straightforward: state the LWD plainly, confirm you can commit to it, and don't over-explain. If pressed on why the notice period is what it is, keep it factual — it's the standard notice at your current employer — and pivot back to availability and enthusiasm for the role rather than dwelling on the current employer.

02 CI/CD — Azure DevOps & GitHub

1 group(s) · 12 questions

PIPELINE DESIGN

- [ ] **Walk through how you'd structure a multi-stage Azure DevOps YAML pipeline for a service that needs to deploy to dev, staging, and production.**

  I'd define one azure-pipelines.yml with a trigger on the main branch, a build stage that compiles/tests and publishes an artifact, then three deploy stages chained with dependsOn, each targeting an Azure DevOps environment (dev/staging/prod). Dev deploys automatically after build; staging and prod sit behind environment approval checks (manual approval gate, possibly a business-hours check for prod). I'd keep the actual deploy steps in a shared template (deploy-template.yml) parameterized by environment name and variable group, so I'm not duplicating logic three times — that consistency is what matters for coordinating deployments across multiple BU teams.

- [ ] **What's the difference between a variable group and a Library secret in Azure DevOps, and when do you use Key Vault linking instead of either?**

  A variable group is a named set of key/value pairs shared across pipelines; a Library secret is just a variable group entry marked secret (masked in logs, not shown in the UI after saving). Both live in Azure DevOps' own storage. Key Vault-linked variable groups pull secrets at pipeline run-time directly from Azure Key Vault instead of storing a copy in Azure DevOps — I'd use that whenever the secret is already centrally managed (DB credentials, API keys) so there's one source of truth and Key Vault's RBAC and audit logging apply, rather than duplicating secret sprawl across pipeline libraries.

- [ ] **How do Azure DevOps environments and approval checks actually enforce a release guardrail — what's under the hood?**

  An environment in Azure DevOps is a named deployment target you can attach checks to: manual approvals, business-hours windows, branch-control checks (only allow deploys from main or a release branch), and Azure Function/REST-based gates for custom logic (e.g. query a change-management system before allowing the deploy). When a stage targets that environment, the pipeline pauses at that stage until every attached check passes — the guardrail is enforced at the platform level, not by convention or a human remembering to check something.

- [ ] **Self-hosted vs Microsoft-hosted agents — how would you decide, and what's the operational cost of self-hosted?**

  Microsoft-hosted agents are zero-maintenance but capped on run time, disk, and network access — fine for standard build/test. Self-hosted agents are needed when you require specific tooling pre-installed, access to an internal/on-prem network (VPN into a client's environment, for instance), specific compute (GPU, larger disk for data-heavy processing), or you're running enough pipeline minutes that hosted-agent cost/queue time becomes a problem. The operational cost is real: you own patching, scaling the agent pool, and security hardening of those machines — so I'd only go self-hosted when hosted agents genuinely can't reach what they need to reach.

- [ ] **What's a GitHub Actions reusable workflow, and why would you use one instead of copy-pasting a workflow into every repo?**

  A reusable workflow is a .yml file called via workflow_call from other workflows, with typed inputs/secrets passed in — effectively a callable pipeline template. I'd use it for anything that needs to be identical across many BU repos (a standard build-test-scan-deploy shape), same reasoning as Azure DevOps pipeline templates: one place to fix a bug or add a security scan step instead of touching N repos.

- [ ] **Your JD lists both Azure DevOps and GitHub — what does coexistence actually look like in practice, and what's the friction?**

  The common pattern is GitHub as source control (or migrating toward it) while Azure DevOps Pipelines still does build/release, using the Azure DevOps GitHub service connection to trigger on GitHub pushes/PRs. The friction is usually around status checks and PR gating — Azure DevOps can post commit statuses back to GitHub, but you lose some of the tight native integration GitHub Actions gives natively, and you end up managing two systems' worth of secrets, permissions, and audit trails. I'd flag this early in any migration conversation rather than let it become tribal knowledge.

- [ ] **How do you handle branching strategy for a platform team supporting multiple BU deployments on different cadences?**

  I lean trunk-based with short-lived feature branches and feature flags for anything that needs partial rollout, because it keeps CI simple — one branch, one pipeline, no drift between long-lived release branches. Where a BU genuinely needs an independent release cadence, I'd use release branches cut from main at a tag with hotfixes cherry-picked back — but I'd treat that as the exception that needs justifying, not the default, since long-lived branches are exactly where merge pain and "which branch is prod on" confusion comes from.

- [ ] **What's a pipeline template in Azure DevOps and how does parameterization work?**

  A template is a separate YAML file with parameters declared, referenced from a calling pipeline via a template reference with parameters passed in. Templates can define steps, jobs, or whole stages. I'd use step templates for common blocks (a standardized scan-and-publish-artifact sequence) and stage templates for entire environment-deploy shapes reused across BU pipelines with just environment name/variable group swapped per call.

- [ ] **How would you implement a canary or blue-green release inside an Azure DevOps pipeline?**

  For blue-green, I'd keep two identical environments (slots, or two parallel deployments) and flip traffic at the load balancer/gateway or via a swap step once the new version passes smoke tests — rollback is just swapping back. For canary, I'd route a small percentage of traffic to the new version, watch error rate/latency against a defined threshold for a bake period, then progressively increase traffic or auto-rollback on a metric breach. I'd be precise about the distinction if asked — blue-green is an all-or-nothing environment swap, canary is progressive traffic shifting; they solve different risk profiles.

What does "green pipeline doesn't mean healthy deployment" mean, and how do you guard against it?

A pipeline can go green because build and unit tests passed, while the actual deployed service is unhealthy — crash-looping, failing readiness checks, or erroring under real traffic that unit tests never exercise. I guard against it with a post-deploy smoke-test/health-check stage that's a required part of the pipeline, plus a bake-time monitoring gate before calling a canary/blue-green rollout "done" — pipeline success should mean "deployed and verified," not just "deploy command exited 0."

How do you keep secrets out of pipeline logs?

Store them as secret variables (masked automatically by Azure DevOps/GitHub Actions when the exact secret value is echoed), never echo or print them directly, and be careful with tools that print full command lines including arguments — a secret passed as a CLI flag can leak even with a "secret" variable if the tool itself logs its invocation. I'd also restrict which pipelines/branches can access a given secret scope, and rotate anything that did leak rather than assume masking caught everything.

A BU team says their deployment pipeline is "too slow" — how do you approach diagnosing that?

I'd measure before changing anything: break the pipeline into stages (checkout, restore/build, test, artifact publish, deploy) and look at duration per stage across recent runs, not just the last one. Common culprits are unnecessary full test suites on every push instead of a fast subset with full regression on merge, no dependency/package caching, serial jobs that could run in parallel, or an underpowered/self-hosted agent pool queuing. I'd fix the biggest measured bottleneck first rather than guessing, and re-measure after each change.

03 Scripting & Automation

1 group(s) · 10 questions

PYTHON / POWERSHELL / GO

When would you reach for Python vs PowerShell vs Go for an automation task in this role?

PowerShell for anything Windows/Azure-resource-heavy where the Az module gives first-class object output I can pipe and filter directly. Python for general automation, API integration, and data-platform-adjacent scripting — it's the common language across DevOps, testing, and data tooling, with the richest library ecosystem for cloud SDKs and log/metric analysis. Go for anything that needs to ship as a single static binary — CLI tools, Kubernetes operators/controllers, or automation that needs to run fast and be distributed without worrying about an interpreter or package versions being present.

Give an example of a Go-based automation tool that would fit a platform-engineering role, and why Go specifically.

A CLI tool that wraps common day-2 operations — say, checking a set of BU deployment health endpoints and posting a status summary — is a good fit for Go: compile once, distribute a single binary to every engineer's machine or CI agent with no runtime dependency, fast enough to run in a tight CI loop without startup overhead. Kubernetes-native tooling (custom controllers, kubectl plugins) is written in Go specifically because it can use client-go directly against the same API the cluster itself uses.

Show me how you'd write a PowerShell script to find and stop all VMs in a resource group that aren't tagged Environment=Production.

Get-AzVM -ResourceGroupName $rg | Where-Object { $_.Tags["Environment"] -ne "Production" } | ForEach-Object { Write-Host "Stopping $($_.Name)"; Stop-AzVM -ResourceGroupName $rg -Name $_.Name -Force -NoWait }. I'd use -NoWait to fire the stop calls in parallel rather than serially waiting on each one, and I'd log what was stopped (and why) somewhere durable, not just to the console, since a tag-based automation script needs an audit trail if someone asks why their VM is down.

Write a short Python approach to poll a REST health endpoint and exit non-zero if it's unhealthy — a deploy-gate smoke test.

Loop with a small retry count (say 5 attempts, 10s apart): GET the health URL with a short timeout, check status 200 and a body field like status=ok, and sys.exit(0) on success. On repeated failure, print the last error and sys.exit(1). The retry-with-backoff matters more than the happy path — a single failed check right after a deploy is often just the service still starting, and a pipeline that fails on the first probe instead of a few retries will generate false-positive rollbacks.

What's your approach to idempotency in automation scripts — why does it matter for VM image or deployment automation?

An idempotent script produces the same end state no matter how many times it runs — running it twice shouldn't create a duplicate resource or error out. This matters because pipelines retry: a transient network blip mid-run shouldn't leave you with two copies of a resource or a half-tagged VM. I write with checks-before-create, or use declarative tools (Terraform, Bicep, ARM) that are idempotent by design rather than imperative create-calls that aren't.

How would you structure a Python automation script that needs to run as part of a scheduled Azure DevOps pipeline vs. as an ad-hoc CLI tool?

I'd separate the logic (a function library) from the entry point — one main() that parses CLI args/env vars for ad-hoc use, and the same functions called from a thin pipeline-triggered wrapper reading its config from pipeline variables instead of CLI flags. That way the core logic gets unit-tested once and reused in both contexts, rather than pipeline-only and CLI-only versions drifting apart.

What logging/exit-code conventions do you follow so a script's failure is actually actionable inside a CI pipeline?

Non-zero exit code on any real failure — never swallow an exception and exit 0. Structured, leveled logging (info/warn/error) to stdout/stderr so it's captured in pipeline logs automatically, with the actual error message near the top of the output, not buried in a stack trace with no context. For anything long-running, periodic progress logging so a stuck step is visible as "hung," not silently timing out with no signal.

How do you handle secrets inside a Python or PowerShell automation script rather than hardcoding them?

Never hardcode — pull from environment variables injected by the pipeline (from a Key Vault-linked variable group), or call Key Vault directly at runtime using managed identity so the script never sees a long-lived credential at all. In PowerShell that's Get-AzKeyVaultSecret with the pipeline's already-authenticated managed identity; in Python it's the Key Vault SDK with DefaultAzureCredential.

What testing do you actually write for infrastructure/automation scripts, given they're harder to unit-test than application code?

I separate pure logic (parsing, decision-making, transformation) from side-effecting calls (the actual cloud API call) so the pure logic is unit-testable without mocking a cloud SDK. For side-effecting parts, I mock the SDK client and assert the right call was made with the right parameters. For anything that provisions infrastructure, I also lean on terraform plan / what-if as a pre-apply sanity check, which is a cheap form of integration testing.

A Go CLI tool you wrote is used by multiple BU teams — how do you version and distribute it so teams don't get breaking changes unexpectedly?

Semantic versioning with tagged releases, published with prebuilt binaries per OS/arch rather than "clone and build yourself." Breaking changes bump the major version, and I'd keep the previous major version's binary available for a deprecation window rather than yanking it — multiple teams pinning to different versions is normal; the failure mode to avoid is a silent behavior change under an unchanged version number.

04 VM Image Automation

1 group(s) · 9 questions

PLATFORM RELIABILITY & GOLDEN IMAGES

The JD calls out "automated VM image creation for compute workloads" — what's your mental model for how that pipeline works end to end?

Source → customize → validate → distribute → deploy. Start from a base (marketplace image, an existing managed image, or a prior gallery version), apply customizers (install agents, apply patches, run hardening scripts, drop in monitoring tooling), validate the result (boot test, basic smoke checks), then distribute the built image as a new version into a shared image gallery, and finally roll it out to compute — ideally with automatic OS-upgrade policies so new instances pick up the latest golden image without a manual step.

What's Azure VM Image Builder (AIB), and how does it differ from rolling your own Packer pipeline?

AIB is Microsoft's managed image-build service, built on top of HashiCorp Packer under the hood — you describe the build declaratively in a template (source, customizers, distribute targets) and AIB manages the transient build infrastructure for you, so there's no VM or storage account to stand up and tear down yourself. Rolling your own Packer pipeline gives more control but means you own that transient infrastructure and its security/lifecycle. I'd default to AIB inside Azure since it's a free service (you only pay underlying compute/storage/network) and integrates directly with Azure Compute Gallery for versioning and distribution.

What's Azure Compute Gallery (formerly Shared Image Gallery), and why not just keep a managed image or a snapshot?

Compute Gallery is a versioned, regionally-replicated image repository — every build becomes a new version under an image definition, with replication to whichever regions need it and RBAC-based sharing across subscriptions. A plain managed image or snapshot is a single unversioned artifact with no built-in replication or lifecycle — fine for a one-off, but the wrong foundation for automatically rebuilding a golden image on a schedule and rolling it out globally, which is what this JD line is really asking for at scale.

How would you trigger an automatic rebuild of the golden image when the base marketplace image gets a security patch?

AIB supports automatic image creation triggers tied to a source update — when the base image you're building from gets a new version, that can kick off a rebuild of your golden image automatically, which then chains into the Compute Gallery version and, with VM Scale Set automatic OS-image-upgrade policies, rolls forward to running instances with no manual step. I'd still put a validation gate in that chain — automatic doesn't mean unverified — so a bad upstream patch doesn't propagate to multiple regions before anyone notices.

Two teams end up with drifted "golden" images that were built by hand at different times — how do you fix the process, not just the immediate images?

The root cause is manual image-building — SSH in, install tooling, snapshot, call it golden — because every hand-built image starts drifting from the next one immediately. I'd move both teams onto one shared, pipeline-driven image template (parameterized customizers per team's needs) so there's one source of truth for what's on the baseline, with per-team differences expressed as parameters/layers on top of a common base rather than two independently-maintained golden images.

What's the risk with someone deleting "the old image version" from a shared gallery, and how do you protect against it?

If a scale set or running fleet is still referencing that version, deleting it can break new instance creation or scale-out at the worst possible time — during an incident when you actually need to scale. Azure Compute Gallery has accidental-deletion protection you can enable on image versions still in use; more broadly, I'd treat image version deletion as a guarded, deliberate action rather than routine cleanup.

How does platform reliability for compute workloads differ from reliability for a single application?

Application reliability is about one service's SLOs — latency, error rate, availability. Platform reliability for compute is the layer underneath: is the golden image current and secure, do scale sets have healthy autoscaling and automatic instance repair, is capacity provisioned ahead of demand for compute-heavy workloads, and can the platform absorb a bad deploy or a regional issue without every application team having to reinvent that resilience individually. I'd frame my role as reducing that burden for every team building on top, not just keeping one app up.

How would you validate a newly built VM image before distributing it, beyond "it booted"?

A validation stage that runs actual health checks against the built image before it's promoted to a gallery version — confirm required agents/services are running, run a smoke test of whatever workload the image is meant to support, check patch/CVE status against policy, and confirm boot time and basic performance haven't regressed versus the previous version. "It boots" is necessary but nowhere near sufficient.

How do you handle Windows vs Linux differences in an automated image pipeline supporting mixed compute workloads?

The pipeline shape (source → customize → validate → distribute) is the same, but customizer types differ — PowerShell/DSC and Windows Update customizers for Windows, shell scripts and package-manager updates for Linux — and validation differs too (sysprep/generalization on Windows before capture, cloud-init readiness on Linux). I'd keep them as separate image definitions in the same Compute Gallery rather than forcing one template to branch internally, since customization and patching cadence for the two OS families are genuinely different operational concerns.

05 BU Deployments & Hypercare

1 group(s) · 8 questions

COORDINATION & PRODUCTION SUPPORT

What does "coordinate BU deployments" actually involve day to day, in your experience?

Aligning a deployment window across teams that don't report to each other and don't share a release calendar by default — confirming code is ready (build green, tests passed, approvals in), confirming the target environment is deployable (no other change in flight, no active incident), communicating the window to stakeholders, and having a clear rollback plan agreed before the deploy starts, not improvised after something breaks. The coordination overhead is usually the actual hard part, not the technical deploy step itself.

What is "hypercare," and how do you structure it after a go-live?

Hypercare is a defined, heightened-support window right after a release or migration — closer monitoring, faster response SLAs, and often a dedicated bridge/war-room presence — before the team steps back down to normal on-call cadence. I'd structure it with an explicit start/end date, named owners per shift, a lower escalation threshold than normal, and a daily check-in to review what came in and whether the exit criteria for ending hypercare are actually met.

A production issue surfaces during hypercare that traces back to a config difference between staging and prod — how do you handle it in the moment vs. afterward?

In the moment: mitigate first — if there's a quick config fix or a rollback available, take it, rather than root-causing live while the issue is ongoing. Afterward: the real fix is closing the staging/prod parity gap that let this happen — if config drift wasn't caught before go-live, that's a gap in pre-deploy validation (a diff check, an environment-parity gate), and I'd add that check rather than just patch this one instance and move on.

How do you do "model validation" as part of a deployment — what does that phrase mean in a DevOps/SRE context versus a data-science one?

In a pure data-science sense it's usually accuracy/precision/recall against a held-out dataset — not my job to own. In the DevOps/SRE sense this JD is pointing at, it more likely means validating a deployed model artifact behaves correctly in production infrastructure terms: does the serving endpoint respond within SLA, does the model version match what was approved, does a canary of the new model's outputs look sane against a baseline before full rollout. I'd ask directly which of these — or both — the team means, rather than assume.

How do you keep multiple BU teams from stepping on each other during a shared deployment window?

A shared deployment calendar with visibility across teams, change-freeze windows communicated in advance, and — where the platform allows it — deployment locks so two teams can't deploy to overlapping infrastructure simultaneously. I'd also push for each team's changes to be independently deployable (feature flags, decoupled services) rather than needing to be bundled into one big synchronized release, since coordinated big-bang releases are where most of this coordination pain comes from.

Describe how you'd handle a production issue where you don't yet know which of three recently-deployed services is the cause.

Correlate by time first — line up each service's deploy timestamp against when the symptom started; anything deployed after symptoms began is the prime suspect. Check each service's own health signals (error rate, latency, saturation) rather than guessing from the symptom alone, and if I can safely and quickly roll back the most recent suspect without much blast radius, I'd do that to reduce impact while continuing to investigate — mitigate fast, root-cause after.

What's your process for writing a post-incident/hypercare summary that's actually useful to stakeholders, not just a timeline?

Lead with impact and current status (what broke, who was affected, is it resolved), then a short timeline of detection → mitigation → resolution, then the actual root cause once known, then concrete follow-up actions with owners and dates — not vague "we'll investigate further." I keep blame out of it entirely; the point is what the system/process allowed to happen, not who made the change.

How do you balance "move fast on BU deployment asks" against "don't skip the SRE guardrails"?

I don't treat those as opposed — guardrails that are automated (pipeline gates, health checks, canary analysis) don't actually slow down a well-formed deployment; they only slow down a deployment that wasn't ready, which is exactly the one you want slowed down. Where there's genuine tension — a BU wants an exception for a time-sensitive release — I'd make the risk explicit and get a documented sign-off rather than quietly waiving the check, so the decision is visible and owned, not just skipped.

06 Testing & Quality

1 group(s) · 9 questions

AUTOMATED TESTING FRAMEWORKS

How do you structure a test pyramid for a platform/backend service, and what goes at each layer?

Broad base of fast unit tests (pure logic, no network/DB), a middle layer of integration tests (real DB or a test container, real calls between your own services), and a thin top layer of end-to-end/UI tests covering critical user journeys only — E2E tests are slow, flaky-prone, and expensive to maintain, so you want as few as will still catch a real regression. Smoke tests sit outside the pyramid as a separate, very fast post-deploy check.

What's the difference between a smoke test and a regression test, and where does each run in the pipeline?

A smoke test is a small, fast set of checks confirming the basics work at all — service is up, key endpoint responds — run immediately post-deploy as a gate before traffic ramps. A regression suite is broader and slower, re-verifying previously-working functionality hasn't broken; it typically runs pre-merge or on a scheduled/nightly cadence rather than blocking every deploy, because running the full suite on every push would make the pipeline too slow to be useful.

How would you approach UI test automation for a platform that multiple BU teams build against — what tooling and what to actually automate?

I'd lean toward tools with good CI integration and low flake rates — Playwright over older Selenium-based stacks where possible, given its built-in auto-waiting and generally more reliable headless CI runs. On what to automate: critical-path journeys only (login, core workflow completion) rather than every screen state, since UI tests are the most expensive layer to maintain — anything checkable at the API/integration layer should be, reserving UI coverage for what genuinely can't be checked lower down.

A regression suite is flaky — passes sometimes, fails sometimes with no code change. How do you approach fixing that instead of just re-running it?

I don't treat "just re-run it" as a fix — a flaky test that gets re-run until green trains the team to ignore failures, which eventually hides a real one. I'd isolate the specific flaky test(s), look for the usual causes (timing/race conditions, shared test state/ordering dependency, an unreliable external dependency), fix or quarantine it into a non-blocking suite while it's being fixed, and track flake rate as a metric so it doesn't silently erode trust in the suite.

How do you decide what needs a unit test versus what's better covered by an integration test?

Unit test anything with real branching logic or business rules testable in isolation — pure functions, validation logic, calculations. Integration test anything where the value is confirming two things actually work together — your code against a real (or realistic test-container) database, or two of your own services talking to each other — because mocking that boundary in a unit test can pass while the real integration is broken.

What does "shift-left" testing mean in practice, and how would you apply it to a CI/CD pipeline you own?

Moving quality checks as early in the pipeline as possible rather than only catching problems late — running linting/unit tests on every push, static/security scanning at PR time rather than pre-release, giving developers fast local feedback (pre-commit hooks) before code even reaches CI. In a pipeline I own, I'd make sure fast checks (lint, unit test, basic security scan) run first and fail fast, so a developer gets feedback in minutes, not after a 20-minute full run.

How would you set up automated testing for infrastructure changes (Terraform/Bicep), not just application code?

terraform plan (or Bicep what-if) in CI as a required PR check, so reviewers see the actual diff before approving — the equivalent of a unit test for infra. For higher-stakes modules, tools like Terratest let you spin up real infrastructure in a test account, assert on it, and tear it down, closer to an integration test; I'd reserve that for critical modules rather than every change given the cost/time.

What metrics would you track to know whether your testing strategy is actually working, not just present?

Escaped-defect rate (bugs found in production that should've been caught earlier), flake rate of automated suites, pipeline/test duration trend (is the safety net getting so slow people start skipping it), and code coverage as a secondary signal — useful for spotting genuinely untested areas, but not a target in itself, since that incentivizes shallow tests written to hit a number rather than meaningful ones.

How do you handle test data for integration tests against something like a subsurface/data-platform backend without touching real production data?

Synthetic or anonymized fixture data seeded into a dedicated test environment or ephemeral test container per run, never a copy of real production data unless properly de-identified and approved for that use — especially relevant for a regulated, large enterprise where data-handling policy is likely strict. I'd version the fixture data alongside the tests so expected results don't silently drift from what the fixture actually contains.

07 SRE — Monitoring & Incidents

3 group(s) · 13 questions

SLI/SLO & OBSERVABILITY

☐ **How do you define SLI, SLO, and error budget, and how do they relate to each other?**

An SLI is a measured indicator of service behavior (e.g. percentage of requests under 300ms, or successful/total request ratio). An SLO is a target for that SLI over a window (99.9% success over 30 days). The error budget is the inverse — the allowed amount of unreliability — and it's what you spend decisions against: burn budget fast, slow down on risky releases; have budget to spare, ship faster. The value is turning "be reliable" into a number you can trade off against.

☐ **What's the difference between monitoring and observability, in your own words?**

Monitoring is watching known signals for known failure modes — dashboards and alerts for things you predicted could go wrong. Observability is having enough structured signal (metrics, logs, and especially traces) that you can answer a question you didn't predict in advance. Monitoring tells you something's wrong; observability helps you find out why when it's a novel shape of problem.

☐ **Explain the RED and USE methods and when you'd use each.**

RED (Rate, Errors, Duration) is request-centric — for services: how many requests, how many failing, how long they take. USE (Utilization, Saturation, Errors) is resource-centric — for infrastructure/compute: how busy, how much queued work, how many errors. For compute-heavy subsurface-style workloads specifically, USE on CPU/memory/queue depth is probably the more telling signal, since the workload profile is closer to batch/HPC than request-response.

INCIDENT MANAGEMENT

☐ **How do you design a burn-rate alert instead of a simple threshold alert, and why is it better?**

A simple threshold alert (error rate > 1%) fires the same whether you're burning your monthly error budget in an hour or over three weeks. A burn-rate alert compares your current error rate against the rate that would exhaust the budget by window-end, at multiple time horizons (a fast 1-hour window paired with a longer 6-hour window to avoid alerting on a brief blip) — so you get paged urgently only when actually on track to breach the SLO, and a lower-urgency signal for slow, sustained degradation.

☐ **Walk through how you'd triage a P1 incident from the moment you're paged.**

Acknowledge immediately so the page doesn't escalate/duplicate, quickly assess actual user impact to confirm severity, declare the incident formally if it meets the bar (opens a bridge, starts the timeline), and focus first on mitigation — rollback, failover, scale out — before deep root-causing. I loop in the right people early rather than solo it, and keep a running timeline as I go rather than reconstructing it afterward.

☐ **What's the difference between Incident Management, Problem Management, and Change Management, and why does the distinction matter?**

Incident Management restores service fast — mitigate now, root-cause later. Problem Management is the follow-up work to find and fix the underlying cause so the same class doesn't recur. Change Management governs how changes get approved and rolled out in the first place, ideally reducing how often Incident Management gets invoked at all. Conflating them slows down incident response — you don't want someone insisting on full root-cause analysis while a P1 is still actively impacting users.

☐ **What's a blameless postmortem, and how do you actually run one without it turning into a blame session anyway?**

A blameless postmortem treats the incident as a systems/process failure, not an individual failure — the gaps that let a bad change reach production are the subject, not the person who pushed it. I keep the language behavioral and factual in the writeup ("the pipeline had no pre-prod validation step for this change type" rather than "X forgot to test"), and actively redirect if the conversation drifts toward fault rather than what's fixable.

☐ **How do you reduce alert fatigue on a team that's drowning in noisy pages?**

Audit what's actually firing — I've cut alert noise by 40% this way before, by tying every alert to whether it required real human action; anything that fires but never leads to action gets deleted, downgraded to a dashboard, or fixed at the source. I'd also consolidate related alerts into composite alerts so several related signals firing together is one incident, not five separate pages, and route by actual urgency instead of everything hitting the same pager.

☐ **What's the difference between "Running" and "Ready" for a Kubernetes pod, and why does that distinction matter for incident response?**

Running means the container process has started; Ready means it's passed its readiness probe and can actually serve traffic — a pod can be Running but not Ready, and the Service correctly excludes not-Ready pods from its endpoint list. During an incident, seeing "Running" isn't enough reassurance — I'd check Ready state and endpoint membership specifically before concluding a pod is actually serving traffic.

☐ **How would you approach capacity planning for a compute-heavy, bursty workload like subsurface/seismic processing, versus a steady web-traffic service?**

Steady web traffic capacity planning is largely a predictable daily/weekly curve with growth headroom. Bursty HPC-style compute is closer to queue-and-scale: provision for a baseline, but design for elastic scale-out (scale sets, spot/low-priority compute where the workload tolerates interruption) that absorbs the burst rather than sizing permanently for peak. I'd track queue depth and time-to-start-processing as the leading indicator that more capacity is needed, rather than waiting for a utilization threshold on already-running instances.

- [ ] **What's your approach to setting up dashboards so an on-call engineer can actually use them at 2 AM, not just admire them?**

  One top-level dashboard per service showing RED/USE at a glance with clear "is this normal" reference lines, not twenty panels requiring interpretation under pressure — the goal is "tells me in 10 seconds if this is the problem," with drill-down dashboards one click away. I've built dashboards using a base-search-plus-post-process-search pattern specifically so the expensive query runs once and multiple panels reuse it, keeping the dashboard fast enough to actually be usable live.

- [ ] **How do you test that your alerts actually work, before you're relying on them in a real incident?**

  Deliberately trigger the condition in a non-prod environment (or safely in prod if low-risk) and confirm the alert fires, routes to the right place, and the runbook context is actually correct and current — an alert that's never been tested is a hope, not a guarantee. I'd also periodically audit existing alerts the same way, since underlying dashboards/queries can silently break (a field rename, a dead data source) without the alert itself erroring.

ITSM & METRICS

- [ ] **What's the difference between MTTA, MTTR, and MTTD, and which one matters most to argue for improving first?**

  MTTD (time to detect) is how long between the problem starting and someone/something noticing. MTTA (time to acknowledge) is detection to a human engaging. MTTR (time to resolve/recover) is the full arc to restoration. I'd argue for improving MTTD first when it's the biggest gap — you can't fix what you haven't noticed — which is exactly the shape of a real result I've delivered before: a 35% MTTD reduction from better detection instrumentation on a prior Azure data platform.

08 Backend & Microservices

1 group(s) · 7 questions

APIS, RESILIENCE & DESIGN

☐ **What do you look for when reviewing an API design from a platform-reliability lens, not a pure functionality lens?**

Idempotency on anything that mutates state, sensible timeouts and retry/backoff behavior for downstream calls, clear error responses with actionable status codes rather than a generic 500 for everything, and whether the API exposes health/readiness endpoints my monitoring can actually use. I also look at whether it degrades gracefully — does a dependency being slow take the whole service down, or fail just that feature and keep serving the rest.

☐ **How do you approach service-to-service resilience in a microservices setup — what patterns actually matter?**

Circuit breakers (Resilience4j-style) so a struggling downstream doesn't cascade into every caller, bulkheads/connection pool limits so one slow dependency can't exhaust the thread pool for everything else, and sensible timeouts everywhere — a call with no timeout is the single most common cause of a slow dependency turning into a full outage. Retries should be bounded and jittered, not naive immediate retries, which can turn a brief blip into a self-inflicted thundering herd.

☐ **What's the difference between a synchronous REST call and an event-driven/async pattern between services, and how do you decide?**

Synchronous REST is simpler to reason about and debug but couples the caller's availability to the callee's — if the downstream is slow or down, the caller is affected immediately. Event-driven decouples that — the caller publishes and moves on — better for workflows that can tolerate delay and for load-leveling bursty work, at the cost of more operational complexity (message durability, ordering, dead-letter handling).

☐ **How would you design health-check endpoints for a microservice so they're actually useful for Kubernetes and monitoring?**

Separate liveness (is the process alive, restart me if not) from readiness (am I able to serve traffic right now, don't route to me if not) rather than one combined endpoint — conflating them means a slow dependency can cause Kubernetes to kill and restart a pod that was actually fine, just temporarily not ready. Readiness should genuinely check critical dependencies without being so heavy it becomes a performance problem itself.

☐ **What's a good rate-limiting/throttling strategy for an internal API used by multiple BU teams?**

Per-consumer quotas (API key or service-identity based) rather than one global limit, so one team's spike doesn't starve everyone else, with a token-bucket or sliding-window algorithm so reasonable bursts aren't punished as harshly as sustained overuse. I'd return a clear 429 with a Retry-After header rather than silently dropping requests, so consuming teams can build correct backoff behavior.

☐ **How do you think about API versioning for a platform other teams build against, so you can evolve it without breaking them?**

Additive changes (new optional fields, new endpoints) don't need a version bump. Breaking changes get a new version with the old version kept alive on a published deprecation timeline, not pulled the moment the new one ships — multiple consumers pinning to different versions at different times is normal, not an edge case.

☐ **What's your approach to structured logging/tracing across microservices so you can actually follow a request across service boundaries during an incident?**

A correlation/trace ID generated at the edge and propagated through every downstream call, logged with every log line each service emits, so I can pull every log line and span across every service for one request. I'd lean toward OpenTelemetry specifically because it's vendor-neutral — instrumentation doesn't lock into one backend, which matters given this JD spans Azure DevOps/GitHub/Google Cloud rather than one fixed stack.

09 Data Platform & AI Enablement

1 group(s) · 7 questions

AI-READY DATA FOUNDATIONS

The JD mentions "contribute to AI-ready data platforms" — what does that mean from a DevOps/SRE seat rather than a data-engineer seat?

My contribution isn't building the models or the pipelines' business logic — it's making sure the platform underneath is reliable and well-structured enough for data/ML teams to build on: CI/CD for data pipeline code, monitoring pipeline health and data-freshness/SLA rather than just infrastructure uptime, access control and audit logging on data platform resources, and repeatable environment provisioning so a data-science team isn't blocked waiting on manually-created infrastructure.

How would you monitor a data pipeline differently from a typical application service?

Application monitoring is mostly request-level health in near-real-time. Data pipeline monitoring needs to also cover data freshness (did the expected data arrive on schedule), volume anomalies (unexpectedly low or high row counts, both suspicious), schema drift, and job success/failure at the batch level. I'd alert on "pipeline didn't run" and "pipeline ran but data looks wrong" as two distinct failure classes, since they need different responses.

What's your understanding of a large-scale industry data platform standard relevant to subsurface/energy data, and would you claim deep expertise in it in this interview?

I'd be direct that I know OSDU (Open Subsurface Data Universe) exists as an industry-standard, cloud-native data platform for E&P data that major operators including Chevron are members/contributors of — but I wouldn't overstate hands-on depth I don't have. I'd frame it as: I understand the shape of the problem — unifying siloed subsurface data behind a common platform so AI/analytics can actually use it at scale — and I'm confident picking up the specific tooling quickly given my Azure data-platform background, rather than claiming OSDU-specific expertise I haven't actually built.

How do you approach access control and audit logging for a data platform handling business-critical, possibly regulated data?

Least-privilege RBAC scoped to what a role actually needs, service principals/managed identities for automation instead of shared credentials, and audit logging turned on and actually reviewed, not just collected and ignored. For a major regulated enterprise like Chevron I'd assume this is taken seriously and ask directly what their existing IAM/data-governance model is rather than propose a generic one from scratch.

What's the difference between enabling a data platform for AI/analytics versus just standing up storage and compute for it?

Storage and compute is necessary but not sufficient — "AI-ready" implies well-structured, discoverable, quality-checked data (schemas enforced, lineage tracked), reasonable performance for the access patterns AI/analytics workloads actually use, and repeatable, self-service environment provisioning so a data-science team can get a working environment without a multi-week infrastructure request cycle.

How would you support "model validation" and rollout for an ML model from a platform perspective, without owning the model's accuracy?

I'd provide the plumbing: deploy a new model version alongside the old one (shadow or canary), route a slice of real traffic/data to it, compare its outputs against the current production model or ground truth, and gate full rollout on that comparison passing agreed thresholds — plus fast, low-risk rollback to the previous model version if the new one underperforms. The accuracy threshold itself is the data-science team's call; making that comparison possible safely in production is the platform's job.

What's your approach when a data platform SLA is breached (data arrived late) but the underlying infrastructure was healthy the whole time?

I'd treat "infrastructure healthy" and "SLA met" as separate signals — the cause is more likely upstream (a source system was late, a dependent job's retry pushed timing back, an unexpectedly large data volume slowed processing) than the platform itself. I'd trace the pipeline's stage-by-stage timing to find where the delay entered, same instinct as diagnosing a slow CI/CD pipeline — measure each stage rather than guessing.

10 Cloud — Azure, GCP & Multi-cloud

1 group(s) · 8 questions

CROSS-CLOUD FUNDAMENTALS

- [ ] **Your JD lists Azure DevOps/GitHub/Google Cloud together — how do you talk about being an Azure-certified engineer being asked to work across GCP too?**

  Honestly: my deepest, certified experience is Azure (AZ-104/SC-300/SC-100), and I've built AWS experience at scale too, so I'm used to translating cloud concepts across providers rather than being locked into one vendor's mental model. GCP's core primitives (Compute Engine, IAM, Cloud Build, GKE) map closely enough to Azure/AWS equivalents that I'd expect to be productive quickly, and I'd say that directly rather than overclaim GCP depth I don't have.

- [ ] **What's the Azure equivalent of GCP's Cloud Build / AWS CodeBuild, and how would you map CI/CD concepts across the three?**

  Azure Pipelines is the direct equivalent — all three are "define build/test/deploy steps as pipeline-as-code, triggered on a commit." Triggers, stages/jobs, artifacts, and environment-scoped secrets transfer directly; what differs is mostly syntax and where secrets/identity live (Key Vault + managed identity, GCP Secret Manager + service accounts, AWS Secrets Manager + IAM roles). I'd expect the biggest ramp-up cost in a new cloud to be identity/IAM model specifics, not pipeline logic.

- [ ] **How does Azure Managed Identity compare to a GCP service account or an AWS IAM role, conceptually?**

  All three solve the same problem — letting a resource authenticate to other cloud services without a long-lived credential sitting in code or config. Azure Managed Identity attaches an identity directly to a resource with Azure AD issuing short-lived tokens automatically; GCP service accounts are a bindable identity with keys or, preferably, workload identity federation; AWS IAM roles work via instance profiles or assumed-role tokens. The principle is identical across all three: prefer the platform-native identity mechanism over a stored credential, always.

- [ ] **What's your experience with Azure Synapse Analytics, and how is that relevant to a data-platform-enablement JD?**

  I worked on a large Azure Synapse Analytics platform build for a major client — direct experience with a large-scale Azure data platform (unified analytics, big-data processing, pipeline orchestration), which is the closest prior experience I have to "AI-ready data platform enablement," even though the domain there wasn't subsurface/energy.

- [ ] **What's a Landing Zone in Azure, and why does it matter for a platform-engineering role supporting multiple BU teams?**

  A Landing Zone is a pre-architected, governed environment — subscription structure, networking, identity, policy, and management-group hierarchy — that new workloads land into with guardrails already in place, rather than every team standing up infrastructure from a blank subscription with its own ad-hoc security and networking decisions. For multiple BU teams deploying onto shared compute platforms, a well-designed landing zone is what lets each team move fast without becoming its own governance/security liability.

- [ ] **How do you approach cost management (FinOps) across cloud resources you're responsible for?**

  Tag everything by team/environment/cost-center from creation, enforced via policy rather than convention, since untagged resources are unattributable cost. Right-size compute based on actual utilization data rather than initial guesses, and for bursty compute-heavy workloads specifically, look hard at spot/low-priority VMs or scale-to-zero patterns where the workload tolerates interruption — that's usually where the biggest waste sits, permanently-on compute sized for peak, idle most of the time.

- [ ] **What Azure networking fundamentals would you expect to be tested on given your AZ-104, and how would they apply here?**

  VNet peering and hub-spoke topology, NSGs vs Azure Firewall for traffic control, private endpoints for keeping PaaS traffic off the public internet, and DNS resolution across peered/hybrid networks. For a role sitting between BU teams and an established corporate network, I'd expect hybrid connectivity (ExpressRoute/VPN into on-prem or another cloud) to matter, given the "subsurface compute platforms" language suggests integration with existing HPC or on-prem systems.

- [ ] **How would you approach learning GCP specifics quickly if that's the gap the interviewer probes hardest?**

  I'd say directly that I'd map GCP's IAM, VPC, and Cloud Build/Deploy concepts against what I already know cold in Azure — same underlying problems, different naming and console — and that my actual ramp risk is low because the JD's core ask (CI/CD, SRE, deployment automation) is provider-agnostic skill, with GCP being wherever a subset of the workload happens to run rather than the whole platform.

11 Networking & Security

1 group(s) · 7 questions

FUNDAMENTALS & SECURE PIPELINES

☐ **What's the difference between an NSG and Azure Firewall, and when would you use each?**

An NSG is stateful L3/L4 packet filtering — allow/deny rules by IP, port, protocol — attached to a subnet or NIC, cheap and simple. Azure Firewall is a managed, centralized L3-L7 firewall with FQDN filtering, threat intelligence feeds, and centralized logging across a hub-spoke topology. I'd use NSGs for basic subnet-level segmentation everywhere, and route egress through Azure Firewall when I need FQDN-based rules or centralized, auditable control over outbound traffic from multiple spokes.

☐ **How do you approach securing secrets and credentials across a CI/CD pipeline end to end, not just "use Key Vault"?**

Key Vault as the source of truth, managed/workload identity for the pipeline to authenticate to it without a stored credential, secret variables (never plain pipeline variables) for anything living in the pipeline layer at all, masked logging, least-privilege access scoped per pipeline, and rotation on a schedule plus immediately on any suspected exposure. "How does a secret get from Key Vault into a running process" is the thing to actually walk through end to end, since that's usually where a shortcut sneaks in.

☐ **What's the security risk of using the :latest tag for container images, and how do you avoid it?**

:latest is mutable — what it points to can change without your deployment manifest changing, so a rollback or redeploy of "the same" manifest can silently pull a different image than what was tested, and it defeats reproducibility for incident investigation. I pin to an immutable tag (a commit SHA or semantic version) and set imagePullPolicy explicitly rather than relying on defaults, so what's deployed is always traceable to an exact, known artifact.

☐ **How would you think about CVE/vulnerability scanning in a CI/CD pipeline for both application code and container/VM images?**

Scan at multiple points: dependency/SCA scanning at PR time, container image scanning (Trivy or similar) as a required pipeline stage before push to a registry, and periodic re-scanning of already-deployed images/running VMs since a CVE can be disclosed after clean deployment. I'd set severity-based gates (block on Critical/High, warn on Medium) rather than either blocking on everything or scanning without enforcement.

☐ **What's the CIDR-math question you'd expect for a VNet/subnet design, and can you walk through an example?**

A /24 gives 256 addresses, minus 5 reserved by Azure (network address, broadcast, and 3 for Azure's own use) leaves 251 usable — so carving an app tier, data tier, and gateway subnet inside a /22 VNet, I'd size into /26s (62 usable each) or individually based on expected host count per tier, always leaving room to grow since resizing a subnet with resources already deployed in it is painful.

☐ **How would you handle a scenario where a secret was accidentally committed and pushed to a Git repo?**

Treat the secret as compromised the moment it's pushed, even if the commit is later removed — rotate it immediately, don't wait to see if anyone saw it. Then clean the git history (BFG Repo-Cleaner or git filter-repo, force-push, address any forks/clones) as the secondary step, since history-cleaning alone with an unrotated secret leaves the actual exposure live. I'd also add pre-commit secret-scanning or a pipeline-level scanner so this is caught before push next time.

☐ **What networking troubleshooting approach would you use if a service in one VNet can't reach a service in a peered VNet?**

Layer by layer: confirm the peering itself is connected on both sides (peering can be misconfigured asymmetrically), check NSG rules on both source and destination subnets/NICs, confirm route tables don't have a conflicting UDR sending traffic elsewhere, then confirm DNS resolution is returning the right IP before assuming it's pure connectivity. I'd isolate which layer is failing with a targeted test (Test-NetConnection or nc) rather than guessing at the whole stack at once.

12 Energy / Subsurface Context

1 group(s) · 7 questions

DOMAIN GROUNDING — VERIFIED, NOT GUESSED

What do you actually know about Chevron's subsurface digital platform work, and how would you frame that knowledge honestly in the interview?

I'd be upfront that this is from public research, not insider knowledge: Chevron has a public, long-running collaboration — sometimes called Triple Crown Partners — with Microsoft and SLB focused on subsurface digital tools, including products like FDPlan (bringing subsurface models together using high-performance computing), DrillPlan, and DrillOps, aimed at cutting the time subsurface professionals spend just locating data rather than doing the actual science. I'd say that context shapes how I'd approach the role — this is clearly a heavy-compute, Azure-centric digital platform effort — but I'd ask the interviewer to fill in the specifics of what I'd actually be supporting.

What's your understanding of why subsurface/seismic workloads are compute-intensive in a way that matters for platform design?

Seismic imaging techniques like full-waveform inversion process enormous volumes of sensor data through iterative, HPC-scale numerical computation to reconstruct a 3D image of the subsurface — genuinely large-scale parallel/batch compute, not a typical request-response web workload. That tells me the platform underneath likely needs to handle large, bursty, long-running batch jobs well — efficient scheduling, scale-out compute, data throughput — more than it needs to optimize for low-latency request handling.

Are you familiar with OSDU, and how would that be relevant here?

Yes, at a conceptual level — OSDU (Open Subsurface Data Universe) is a cross-industry, cloud-native data platform standard for E&P data that Chevron participates in developing alongside other major operators and cloud/vendor partners, aimed at replacing siloed, hard-to-access subsurface data with a common, interoperable platform. If this role's AI-ready data platform work touches an OSDU-based system, platform-engineering fundamentals transfer directly — schema-governed data, access control, reliable ingestion — even without prior OSDU-specific hands-on time.

Chevron has talked publicly about in-house generative AI/ML platforms for exploration — what's your take on supporting that kind of workload as a platform engineer, versus a typical enterprise AI feature?

The public examples I'm aware of — tools that speed up prospecting by pulling together surveys, well logs, seismic analyses, and reports — sound like classic AI-enablement platform work: making previously siloed, manually-assembled data available to a model reliably, at scale, with good data lineage, which is squarely a data-platform-reliability problem, not a model-training problem. I'd support it the way I'd support any production ML system: reliable data pipelines feeding it, safe rollout/validation for model updates, and monitoring that distinguishes "the platform is unhealthy" from "the model's outputs look off," since those need different responders.

How would you handle not knowing Chevron-specific internal tooling/acronyms (like PPO/FDO) in an interview, without it looking like a knowledge gap?

I'd say plainly these are internal names I wouldn't expect to know before joining, ask a clarifying question to understand the workflow's actual shape, and then answer the underlying technical question against what they describe rather than try to bluff familiarity. Interviewers generally read "I don't know that acronym, can you unpack it" as confidence and good judgment, not a gap — pretending to know it and getting the meaning wrong would read far worse.

Why would Chevron specifically want dual-cloud (Azure + GCP) skills for a subsurface-adjacent role rather than pure Azure?

Given Chevron's deep, public Microsoft/Azure partnership for subsurface HPC, I'd expect Azure to be the primary platform for this specific workflow — the GCP mention is more likely about breadth across Chevron's broader technology estate (a large enterprise rarely runs one cloud everywhere) than a signal this specific PPO/FDO workflow is GCP-native. I'd frame my answer the same way either way: strong Azure depth, transferable multi-cloud fundamentals, and I'd confirm directly which cloud this specific workflow actually runs on rather than assume.

How do you show genuine interest in the energy/oil & gas domain in an interview without pretending to have domain expertise you don't have?

I'd point to what I actually did to prepare — reading Chevron's own public materials on their subsurface digital platform work rather than winging it — and frame my interest honestly: I'm drawn to the scale and stakes of the compute problem, huge data volumes and expensive decisions riding on platform reliability, more than claiming geoscience knowledge I don't have. Genuine curiosity backed by a couple of specific, accurate references reads far better than pretending prior expertise.

13 Tricky & Scenario Questions

1 group(s) · 9 questions

REAL-TIME SITUATIONAL QUESTIONS

- [ ] **"Your pipeline is green, the deploy succeeded, but users are reporting errors 10 minutes later. Walk me through what you do."**

  First, confirm real impact — check RED metrics (error rate, latency) for the service directly rather than trusting the report alone. If confirmed, decide fast whether rollback is safe and available — if yes, roll back immediately rather than debugging live, since restoring service beats understanding root cause in the first ten minutes. Only after mitigating do I dig into why a green pipeline let this through — likely a gap between what the smoke test checked and what actually broke, which becomes the process fix afterward.

- [ ] **"You're asked to make an emergency production change outside the normal change window. What do you actually do?"**

  I don't skip the guardrails, I compress the timeline around them — get a fast, documented approval (even a quick message thread with the right approver is better than nothing), make the smallest possible change that fixes the immediate problem rather than bundling in other fixes "while I'm in there," and make sure the change is captured in the normal change record after the fact even if the approval itself was expedited. Emergency doesn't mean untracked.

- [ ] **"Two BU teams both need the shared staging environment for testing at the same time this week. How do you resolve it?"**

  I'd first check whether they actually need the same environment or just think they do — often the real conflict is a shared dependency, not the whole environment. If it's genuinely a scheduling conflict, I'd broker a shared calendar/booking system rather than deciding for them, and push toward the actual fix — ephemeral per-branch or per-PR environments — so this stops being a recurring negotiation.

- [ ] **"A Terraform apply partially failed halfway through — some resources created, some not. What's your process?"**

  Don't panic-apply again immediately — run terraform plan first to see exactly what state Terraform thinks exists versus what's actually in the cloud, since a partial failure can leave state and reality mismatched. Investigate why it failed (a quota limit, naming conflict, transient API error) before retrying, and if state itself looks wrong, use terraform state commands to reconcile rather than blindly re-running apply and risking duplicate or conflicting resources.

- [ ] **"You inherit a system with three different Terraform state files, no clear ownership, and no one's sure which is authoritative. How do you approach it?"**

  Don't touch any of them destructively yet — first inventory what each state file actually manages against real cloud resources to understand overlap and gaps. Then establish ownership and a single source of truth going forward, migrating state carefully with terraform state mv rather than destroying and recreating resources, since recreation on stateful resources can mean real data loss or downtime a state migration avoids entirely.

- [ ] **"An automated golden-image rebuild just rolled a broken image into three regions before anyone caught it. How do you contain it, and how do you stop it happening again?"**

  Contain: use Compute Gallery's version control to point new scale-out at the previous known-good version immediately, and pause the automatic OS-upgrade rollout policy so it doesn't keep propagating. Prevent recurrence: the automation had no validation gate between build and distribute — add one, a real smoke test against the built image before it's promoted to a gallery version, which is exactly the "automated, but not blindly trusted" principle for image automation.

- [ ] **"You're told to cut a corner on testing to hit a hard deadline the business has already committed to externally. How do you handle that?"**

  I don't just refuse or just comply — I make the actual risk visible and quantified (what specifically won't be tested, what's the realistic failure mode and blast radius) and let the people who own the business risk make an informed call, rather than silently deciding either way myself. If they accept the risk with that information, I proceed and document the decision; what I won't do is quietly skip something and let people believe it was fully tested.

- [ ] **"A monitoring alert has been silently broken for three weeks — a dashboard query started returning nothing after a field rename, and nobody noticed until an actual incident had no warning." How do you respond, and how do you prevent a repeat?**

  In the moment, that's a straightforward blameless-postmortem finding — the monitoring itself became a single point of failure with no one watching the watcher. The fix is meta-monitoring: alert on "this alert hasn't fired or evaluated successfully in the expected timeframe" and periodically test-fire real alerts against known conditions, rather than trusting an alert stays correct forever once set up once.

- [ ] **"You disagree with a senior architect's decision on how to structure the CI/CD pipeline for this platform. How do you handle that?"**

  I'd raise the disagreement directly and early, with the specific technical reasoning behind my concern — concrete tradeoffs, not a vague objection. If they hear the reasoning and still want to go their way, I implement their decision fully and professionally rather than under-delivering on something I disagreed with — disagree-and-commit, unless it's a genuine reliability/security red flag, in which case I'd escalate that specifically rather than just going along.

14 Leadership & Behavioral

1 group(s) · 10 questions

BRING-YOUR-OWN FRAMEWORKS

☐ **"Tell me about a time you improved platform reliability with a measurable result."**

Bring your own real example here — you have verified numbers to anchor this: a real 40% alert-noise reduction and 35% MTTD improvement from Azure Synapse platform work, or a real 3-SPOF/45%-recovery-time chaos-engineering result. Structure it as: what was unreliable and how you knew (the baseline metric), what you changed, and the measured after-state — interviewers respond well to a real before/after number, and you have several genuine ones to choose from rather than needing to invent anything.

☐ **"Describe a situation where you had to coordinate a deployment across teams that didn't report to you."**

Bring your own real example — this maps directly to real work you've done coordinating cross-team deployments. Structure: what made coordination hard (competing priorities, no shared calendar, unclear ownership), what you actually did to align them, and the outcome. The specific mechanism you used to get alignment is what differentiates this from a textbook answer — avoid a generic version.

☐ **"How do you approach mentoring or supporting a junior engineer on your team?"**

Bring your own real example if you have one; if not, here's an adaptable framework: pair on real production work rather than assign toy problems, give them ownership of something real with a safety net (review before it ships, not after), and calibrate feedback to be specific and actionable rather than generic praise or criticism. If asked about formal management scope, ground the answer in guild/technical-mentoring language unless you actually have direct reports.

☐ **"Tell me about a time you had to push back on a deadline or scope, and how you handled it."**

Bring your own real example. Framework if needed: state the tradeoff in concrete terms (what specifically is at risk if we don't adjust, not just "it's risky"), propose an alternative (phased delivery, reduced scope, extra resource) rather than just saying no, and let the decision-maker choose with full information rather than unilaterally deciding for them.

☐ **"What does 'automation' actually mean to you as a narrative, beyond 'I wrote a script'?"**

The compelling version isn't "I automated X" — it's what stopped happening because of it: manual toil that used to eat someone's time, a class of human error that used to happen, or a response time that used to be hours and became minutes. Bring your own real example and frame it that way — before-state (the pain), what you built, after-state (measured, if you have the number) — rather than describing the tool itself as the point.

☐ **"How do you track whether an underperforming team member is actually improving?"**

Framework, since this isn't confirmed as part of direct people-management experience: specific, agreed expectations rather than vague "do better," a defined check-in cadence, concrete examples tied to those expectations rather than general impressions, and a clear timeline with what "improved" and "not improved" actually look like decided up front. If there's a real example of tracking your own or a peer's improvement against explicit goals, that's more credible than a purely theoretical answer.

☐ **"What does a typical day look like for you as an SRE/DevOps engineer?"**

Bring your own real day-to-day detail — this is the easiest question to make concrete and genuine, since it's literally your actual work. A believable shape: checking overnight alerts/dashboards first, working through planned pipeline/platform work, responding to whatever team asks come in, and some portion of time on proactive reliability work that doesn't get done if you only ever react — but fill in your own actual balance rather than reciting this structure verbatim.

☐ **"Why are you looking to move roles now, given your current notice period and LWD?"**

Keep this forward-looking and specific to what this role offers rather than negative about the current one — the platform-reliability/SRE scope at enterprise scale, an Azure-heavy stack matching your certifications, and the appeal of the domain if that's genuinely true for you. Confirm the LWD (22 July 2026) plainly if asked, without over-explaining the current employer's notice policy.

☐ **"This is contract-to-hire — how do you show commitment to a role that isn't guaranteed permanent from day one?"**

Treat the contract period as the actual audition it is — deliver real, measurable value early (a quick reliability or process win in the first 60–90 days is more persuasive than any verbal assurance), and be direct that you're evaluating fit both ways, which reads as confidence rather than desperation. Reference your own history of long tenures at prior employers as evidence you're not a flight risk once you find the right fit.

☐ **"Walk me through your career progression and why this Chevron role is the right next step."**

Structure chronologically but end on the throughline, not a list: 14+ years moving from infrastructure/support roles toward reliability-and-platform ownership across banking, a major Azure data platform engagement, and other enterprise clients, with Azure certifications (AZ-104/SC-300/SC-100) added deliberately to formalize that specialization. Frame this role as the next step because it's platform-reliability work at genuine enterprise scale with real technical stakes — not just a lateral title change — and keep specific real details from your own history in the close rather than a generic "I want to grow" line.